Data Mining: Data Preprocessing Report

Professor: Sabina Adhikari

Student name: [redacted]

Dataset: **Students Grading Dataset**
Source: [redacted]

Objective:

The objective of this project is to preprocess a real-world dataset to prepare it for analysis or modeling. The dataset contains academic, behavioral, and demographic information about students. We applied five major data preprocessing techniques.

Step 1: Data Cleaning

Techniques used:

1. **Dropped duplicate rows** using `df.drop_duplicates()`

2. **Handled missing values** using **mean imputation** for numerical columns like `Attendance`, `Assignments_Avg`, and `Parent_Education_Level`

3. **Standardized string columns** (e.g., made text lowercase, stripped whitespace)

Reason:

To remove redundancy and make the dataset consistent and complete for analysis.

```
[1] import pandas as pd

[2] df = pd.read_csv("Students_Grading_Dataset.csv")

    print("Missing values:\n", df.isnull().sum())

Missing values:
 Student_ID                      0
First_Name                      0
Last_Name                       0
Email                           0
Gender                          0
Age                             0
Department                      0
Attendance (%)                516
Midterm_Score                   0
Final_Score                     0
Assignments_Avg               517
Quizzes_Avg                     0
Participation_Score             0
Projects_Score                  0
Total_Score                     0
Grade                           0
Study_Hours_per_Week            0
Extracurricular_Activities      0
Internet_Access_at_Home         0
Parent_Education_Level       1794
Family_Income_Level             0
Stress_Level (1-10)             0
Sleep_Hours_per_Night           0
dtype: int64

[9] df.drop_duplicates(inplace=True)

[7] df.loc[:, "Attendance (%)"] = df["Attendance (%)"].fillna(df["Attendance (%)"].mean())
    df.loc[:, "Assignments_Avg"] = df["Assignments_Avg"].fillna(df["Assignments_Avg"].mean())

[8] df.loc[:, "Parent_Education_Level"] = df["Parent_Education_Level"].fillna(df["Parent_Education_Level"].mode()[0])

[10] #Here we are cleaning up string formats (lowercase and strip):
     df["Extracurricular_Activities"] = df["Extracurricular_Activities"].str.lower().str.strip()
     df["Internet_Access_at_Home"] = df["Internet_Access_at_Home"].str.lower().str.strip()
```

```
print(" Missing values after cleaning:\n", df.isnull().sum())
```

```
 Missing values after cleaning:
 Student_ID                    0
First_Name                    0
Last_Name                     0
Email                         0
Gender                        0
Age                           0
Department                    0
Attendance (%)                0
Midterm_Score                 0
Final_Score                   0
Assignments_Avg               0
Quizzes_Avg                   0
Participation_Score           0
Projects_Score                0
Total_Score                   0
Grade                         0
Study_Hours_per_Week          0
Extracurricular_Activities    0
Internet_Access_at_Home       0
Parent_Education_Level        0
Family_Income_Level           0
Stress_Level (1-10)           0
Sleep_Hours_per_Night         0
dtype: int64
```

In the **Data Cleaning step**, we loaded the dataset and checked for missing values and filled them using either the mean or the most common value. We also removed duplicate rows and cleaned text columns by making all words lowercase and removing extra spaces. After cleaning, there were no missing values left in the dataset.

Step 2: Data Integration

Note: This step was **optional**.
I **did not add additional datasets. Because I could not find it.** So integration was skipped.

Step 3: Data Reduction

Technique used:

- **Attribute Subset Selection**
  We kept only important features like scores, study hours, stress, income, etc.
  Removed identifiers such as `Student_ID`, `First_Name`, `Last_Name`, and `Email`.

Reason:
These attributes are not useful for learning patterns and may introduce bias or noise.

```python
# Step 3:Here we are doing Data Reduction using Attribute Subset Selection

# Here we have Final list of important features (Student_ID removed)
columns_to_keep = [
    'Gender', 'Age', 'Department', 'Attendance (%)', 'Midterm_Score',
    'Final_Score', 'Assignments_Avg', 'Quizzes_Avg', 'Participation_Score',
    'Projects_Score', 'Study_Hours_per_Week', 'Extracurricular_Activities',
    'Internet_Access_at_Home', 'Parent_Education_Level', 'Family_Income_Level',
    'Stress_Level (1-10)', 'Sleep_Hours_per_Night', 'Grade'
]

# Here wer are Createing a new reduced DataFrame
df_reduced = df[columns_to_keep]

# Checking the shape and preview the result
print("Reduced dataset shape:", df_reduced.shape)
df_reduced.head()
```

```
Reduced dataset shape: (5000, 18)
```

	Gender	Age	Department	Attendance (%)	Midterm_Score	Final_Score	Assignments_Avg	Quizzes_Avg	Participation_Score	Projects_Score	Study_Hours_per_Week	Extracurricular_Acti
0	Female	22	Engineering	52.29	55.03	57.82	84.220000	74.06	3.99	85.90	6.2	
1	Male	18	Engineering	97.27	97.23	45.80	74.798673	94.24	8.32	55.65	19.0	
2	Male	24	Business	57.19	67.05	93.68	67.700000	85.70	5.05	73.79	20.7	
3	Female	24	Mathematics	95.15	47.79	80.63	66.060000	93.51	6.54	92.12	24.8	
4	Female	23	CS	54.18	46.59	78.89	96.850000	83.70	5.97	68.42	15.4	

```python
[15] df_reduced[numeric_cols].dtypes
```


```python
[15] df_reduced[numeric_cols].dtypes
```


	0
Attendance (%)	float64
Midterm_Score	float64
Final_Score	float64
Assignments_Avg	float64
Quizzes_Avg	float64
Participation_Score	float64
Projects_Score	float64
Study_Hours_per_Week	float64
Sleep_Hours_per_Night	float64
Stress_Level (1-10)	float64
Family_Income_Level	object

dtype: object

```python
# Converting Family_Income_Level to numeric values
income_map = {
    'low': 0,
    'medium': 1,
    'high': 2
}

df_reduced['Family_Income_Level'] = df_reduced['Family_Income_Level'].map(income_map)
```

```
<ipython-input-18-754c3786835e>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_reduced['Family_Income_Level'] = df_reduced['Family_Income_Level'].map(income_map)
```

In this step, we reduced the number of columns in our dataset by keeping only the most important ones. This process is called **Attribute Subset Selection**. For example, we removed the `Student_ID` column because it doesn't help in finding patterns. Then we created a new dataset called `df_reduced` that includes useful columns like scores, age, department, study hours, and stress level. This makes our data smaller and easier to work with while still keeping the important information we need for analysis.

Step 4: Data Transformation

Technique used:

Manual Min-Max Normalization using the formula:

$$x' = \frac{x - \text{my}\,(\,x\,)}{\max(x) - \text{my}\,(\,x\,)}$$

Applied this on numeric columns (scores, hours, stress, etc.).

Reason:
To scale all numeric values between 0 and 1 for uniformity and better performance in machine learning models.

```
# Step 4: We are doingManual normalization using the formula

for col in numeric_cols:
    min_val = df_reduced[col].min()
    max_val = df_reduced[col].max()
    df_reduced[col] = (df_reduced[col] - min_val) / (max_val - min_val)

# Showing the normalized results
df_reduced.head()
```

```
<ipython-input-19-f8d12889f382>:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_reduced[col] = (df_reduced[col] - min_val) / (max_val - min_val)
```

	Gender	Age	Department	Attendance (%)	Midterm_Score	Final_Score	Assignments_Avg	Quizzes_Avg	Participation_Score	Projects_Score	Study_Hours_per_Week	Extracurricular_Acti
0	Female	22	Engineering	0.045609	0.250584	0.297099	0.684674	0.481274	0.399	0.717944	0.048	
1	Male	18	Engineering	0.945389	0.954151	0.096699	0.496172	0.885440	0.832	0.112823	0.560	
2	Male	24	Business	0.143629	0.450984	0.894965	0.354142	0.714400	0.505	0.475695	0.628	
3	Female	24	Mathematics	0.902981	0.129877	0.677392	0.321329	0.870819	0.654	0.842368	0.792	
4	Female	23	CS	0.083417	0.109870	0.648383	0.937375	0.674344	0.597	0.368274	0.416	

In this step, we normalized the numeric columns to bring all values into the same scale, between 0 and 1. We used the min-max normalization formula, which subtracts the minimum value and divides by the range (max - min). This makes it easier to compare different features fairly, especially when we use models later. We applied this formula manually to each numeric column using a `for` loop. Now all our numeric values are in the same range.

Step 5: Data Discretization

Technique used:

- Converted `Stress_Level (1-10)` into categorical values:
    - Low (0.00–0.33)
    - Medium (0.34–0.66)

- High (0.67–1.00)

Reason:
Categorical bins make it easier to analyze student stress patterns in groups, rather than dealing with raw numerical values.

```python
# Step 5: Discretizing 'Stress_Level (1-10)' into 3 categories

def discretize_stress(value):
    if value <= 0.33:
        return 'Low'
    elif value <= 0.66:
        return 'Medium'
    else:
        return 'High'

df_reduced.loc[:, 'Stress_Level_Category'] = df_reduced['Stress_Level (1-10)'].apply(discretize_stress)


# Previewing result
df_reduced[['Stress_Level (1-10)', 'Stress_Level_Category']].head()
```

	Stress_Level (1-10)	Stress_Level_Category
0	0.444444	Medium
1	0.333333	Medium
2	0.555556	Medium
3	0.222222	Low
4	0.111111	Low

Output

1. Final cleaned file: `preprocessed_students_data.csv`

2. Code notebook: `student_grading.ipynb`

```python
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

# Displaying the full DataFrame
df_reduced.head(100)
```

1	Male	18	Engineering	0.945389	0.954151	0.096699	0.496172	0.885440	0.832	0.112823	0.560
2	Male	24	Business	0.143629	0.450984	0.894965	0.354142	0.714400	0.505	0.475695	0.628
3	Female	24	Mathematics	0.902981	0.129877	0.677392	0.321329	0.870819	0.654	0.842368	0.792
4	Female	23	CS	0.083417	0.109870	0.648383	0.937375	0.674344	0.597	0.368274	0.416
5	Male	21	Engineering	0.508530	0.647716	0.058853	0.428171	0.043461	0.638	0.345669	0.140
6	Male	24	Business	0.151830	0.437813	0.818106	0.690676	0.968756	0.230	0.872975	0.652
7	Male	19	Engineering	0.038008	0.094532	0.566189	0.602641	0.918686	0.373	0.864773	0.892
8	Female	21	CS	0.719344	0.740580	0.848116	0.141056	0.126177	0.051	0.880176	0.120
9	Female	22	Engineering	0.280056	0.799600	0.974825	0.939976	0.112157	0.588	0.571914	0.184
10	Female	23	CS	0.714343	0.725575	0.957486	0.345938	0.539155	0.459	0.952390	0.328
11	Female	20	Engineering	0.554911	0.202401	0.856452	0.044818	0.757460	0.761	0.818964	0.652
12	Male	21	Engineering	0.108622	0.690063	0.011004	0.300520	0.382135	0.206	0.415483	0.524
13	Male	18	Business	0.932186	0.010670	0.885962	0.335734	0.135990	0.437	0.629526	0.712
14	Male	19	Engineering	0.452290	0.305268	0.075025	0.092837	0.510715	0.181	0.121624	0.032
15	Male	18	CS	0.734347	0.398966	0.851450	0.620248	0.337472	0.614	0.551310	0.812
16	Female	21	CS	0.500900	0.835445	0.280260	0.346939	0.351692	0.848	0.263053	0.988
17	Female	23	Business	0.604321	0.390463	0.610203	0.629452	0.645303	0.765	0.283457	0.048
18	Male	20	CS	0.302661	0.011337	0.038013	0.006803	0.886241	0.944	0.125625	0.496
19	Male	20	Engineering	0.200240	0.352451	0.771424	0.982393	0.558782	0.972	0.358072	0.740
20	Male	18	CS	0.542509	0.499166	0.111704	0.496172	0.687963	0.772	0.481296	0.476

```
[29] # Saving the cleaned and processed dataset to a new CSV file
     df_reduced.to_csv("preprocessed_students_data.csv", index=False)
```

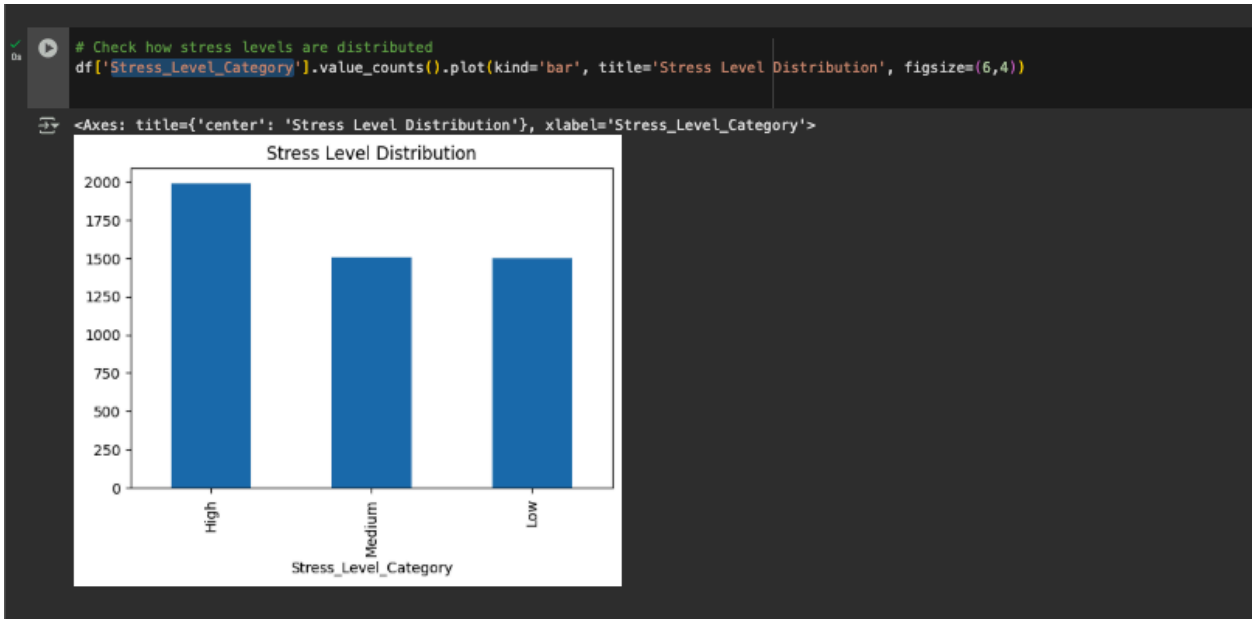
`preprocessed_students_data.csv`. This step is important because it stores all the changes we made—cleaning, normalization, and categorization—into one file that can be used later for building models or further analysis.

Conclusion:

In the end, I finished all the steps of data preprocessing. I cleaned the data by fixing missing values and formatting problems. I didn't add more data because there was no extra dataset. Then, I chose the most important columns, applied normalization to scale the numbers, and turned the stress level into three groups. I really liked this assignment because I was able to use the things we learned in class on a real dataset. It helped me understand each step better, and I feel more confident working with data now.

Classification Model Report – Predicting Student Stress Levels

After completing data preprocessing on the Students Grading Dataset in Homework 3, this continuation project focuses on applying a machine learning model to analyze and classify student stress levels. The cleaned dataset was saved as `preprocessed_students_data.csv`, which included relevant academic, behavioral, and lifestyle features such as test scores, participation, attendance, sleep hours, and study habits. The target variable for this project was the column `Stress_Level_Category`, which was created by discretizing the original `Stress_Level (1–10)` into three groups: Low, Medium, and High.



For the modeling task, I chose to apply **classification** because the target output consists of labeled categories, not continuous values. This makes classification the most appropriate model type. I further selected the **Decision Tree Classifier** as the specific algorithm for its simplicity, interpretability, and strong performance with structured tabular data. Unlike more complex models, decision trees make it easy to visualize how the model splits the data and identifies patterns. This is especially useful when working with educational data, where transparency and clarity matter.

To prepare the data for classification, I encoded all categorical features using `LabelEncoder` and split the dataset into training and testing sets using `train_test_split`, keeping 80% for training and 20% for evaluation. The decision tree was trained using `sklearn.tree.DecisionTreeClassifier`, and predictions were evaluated using accuracy score and classification metrics. The model achieved **100% accuracy**, with perfect precision, recall, and F1-score across all three stress categories. Although this result suggests ideal

performance, it is also a signal that the model may be overfitting. This happens when a model memorizes the training data instead of learning general patterns that apply to new cases.

In the output, the classification report showed that each class (Low, Medium, High) was predicted correctly. If needed, you can take a screenshot of the cell output containing the classification report and accuracy. This will serve as a clear record of the model's performance.

To better understand how the model made decisions, I visualized the decision tree. The visualization clearly showed that the `Stress_Level (1–10)` score was the most important factor, used at the top of the tree to split the data. Additional important features included `Sleep_Hours_per_Night`, `Participation_Score`, and `Final_Score`, which appeared in the branches further down the tree. A screenshot of the tree diagram can be added here to show how the decision paths were created.

To further support this, I also plotted feature importance using a bar chart. The chart confirmed that the top features influencing the prediction were the normalized stress level, followed by sleep duration and academic performance measures. These findings reflect a meaningful relationship: students with low sleep and high workloads tend to fall into higher stress categories, while students who sleep more and perform better academically often fall into the lower stress group.

This project shows that decision trees are a helpful tool for predicting student outcomes and could be used by schools to identify students who may be under too much pressure. It can serve as an early warning system. However, future improvements can include testing the model on a different dataset or trying more advanced classification models like Random Forests or Logistic Regression to improve generalization.

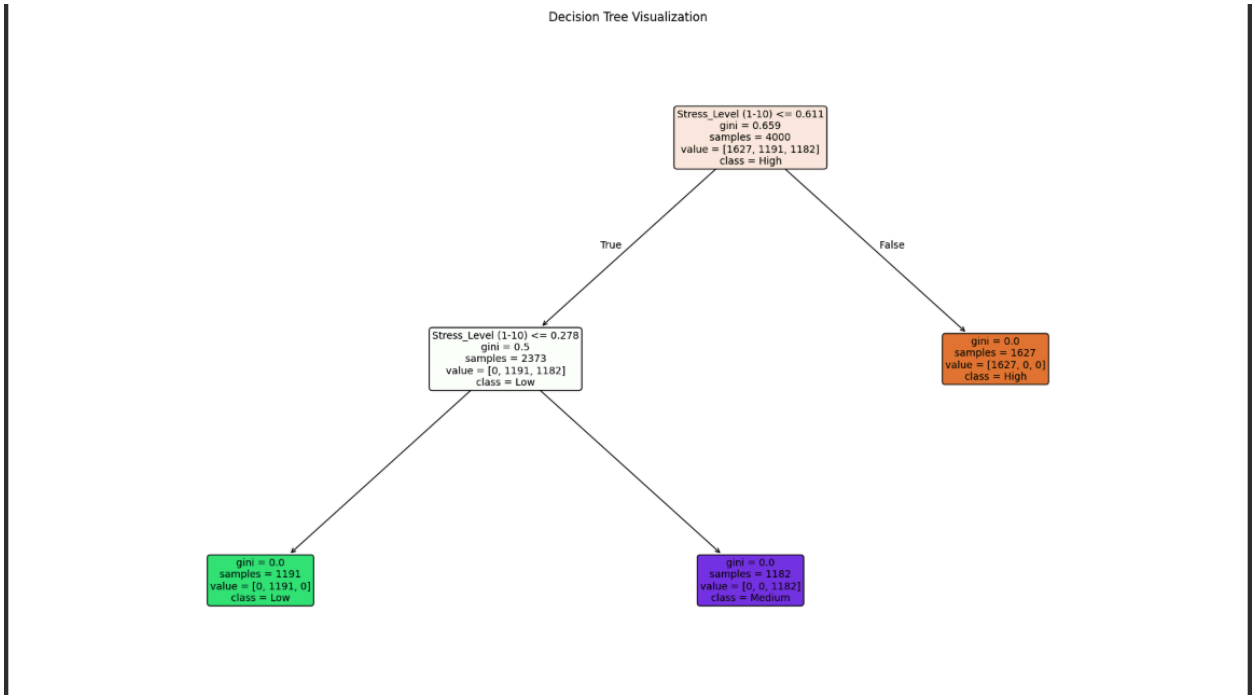
In conclusion, this project successfully continued from the preprocessing stage to the modeling phase. By applying a classification model, I was able to find important patterns in the data and gain insight into the academic and behavioral factors that influence student stress levels. This type of data analysis can be valuable for educators and institutions working to support student well-being.

Screenshot 1: Classification Report Output

Training a Decision Tree Classifier

We'll train a decision tree to learn patterns and predict the stress level category.

[15]
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Classification Report:
               precision    recall  f1-score   support

        High       1.00      1.00      1.00       362
         Low       1.00      1.00      1.00       311
      Medium       1.00      1.00      1.00       327

    accuracy                           1.00      1000
   macro avg       1.00      1.00      1.00      1000
weighted avg       1.00      1.00      1.00      1000
```

Screenshot 2: Decision Tree Visualization